

27.05.2024г.

Лекция 14

Шабини

→ параметричен полиморфизъм

→ темплейтна специализация

sort какви са необходимите функции?

operator < // трябва да се напише, за да няма грешки

от C++20: концепции →

експлицитно деклариране на необходимите функции на даден шаблон.

Умни указатели

→ Обвиващ клас на указател,

не се интересуваме от триенето (не го правим ние)

new ← delete

→ прави го указателя

1) auto_ptr - deprecated // стар е и не се използва

1. Unique_ptr

- точно 1 указател за точно 1 обект

Unique_ptr <A> (A)

(*ptr).x++;

ptr → x++

⑤ → ~Up
извиква ~Aуказател към динамично
зарежен обект A

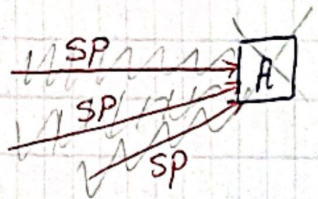
new A(...)

make_unique(<A>(...))

- забранени са K.K и оператор =
- разписани са M.K и M.op =

II. Shared_ptr

- 1 обект, но много указатели



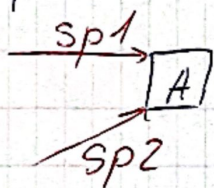
→ при първия указател се заделя обекта

→ при изтриването на последния указател към обекта се изтрива и самия обект

```
Shared_ptr<A>(SP=make_shared<A>(2,3));
```

```
shared_ptr<A>(new A(2,3));
```

```
Shared_ptr<A> sp2=sp
```



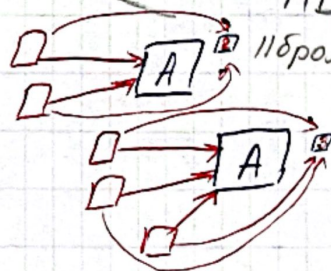
• ще имаме копиране и местене

Как се имплементира триенето?

→ статична член-функция

НЕ!

```
class Sp  
int *refcount  
T* data
```



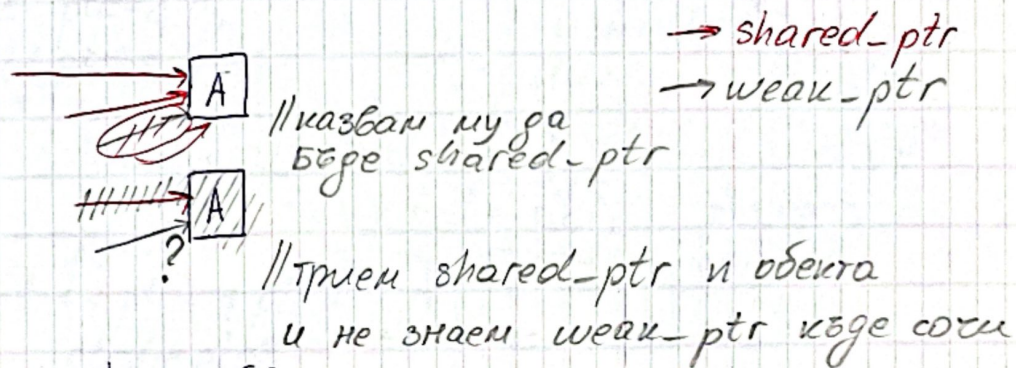
//броят на referance

//в копи конструктора
кроем refcount

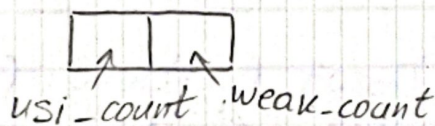
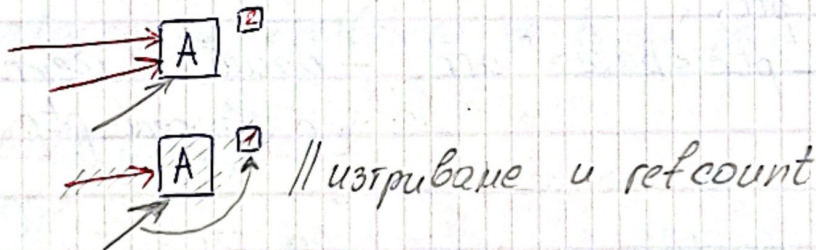
```
~Sp() {  
    (*refcount)--;  
    if ((*refcount)==0) {  
        delete data;  
        delete refcount;  
    }  
}
```


III. Weak_ptr (non-owning ref)

- Указател към обект, който е менижиран от `shared_ptr`
- ! → Не влияят на триенето
- Възможно е да сме насочени, към обект, който вече е изтрит.
- Възможно е промотиране в `shared_ptr`,



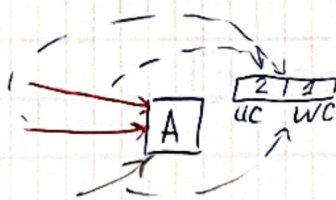
⚠ Weak_ptr трябва да има проверка дали обектът е още жив.



`use_count` - брой `shared_ptr`

`weak_count` - брой `weak_ptr` + 1/0

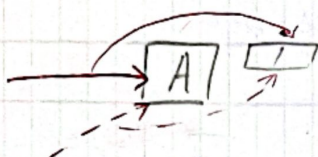
1, ако `use_count ≥ 1`
0, ако `use_count = 0`



→ Кой трие обекта A? и Кога?
use_count=0

→ Кога се трие броята?
weak_count=0

~~weak_ptr няма нищо
общо с обекта A.
// weak_ptr е свързан
с shared_ptr~~



// operator → броя тип T& или T*
спестява (sp →) → x++
стаба
sp → x++

Хетерогенен контейнер

Base** ptr;

Shared_ptr <Base>* arr; - масив от обекти
с shared_ptr

// нужно е да имаме Big 6,
заради копирането, може да станат
shadow copy

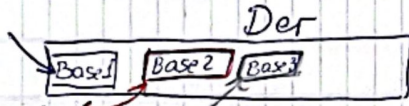
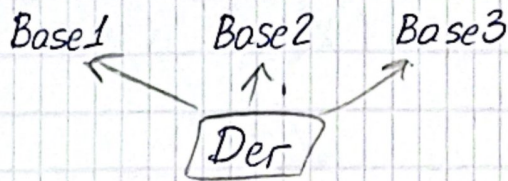
домашно:

polymorphic_ptr
Изпит

// така че да
направим хетерогенен
контейнер

Polymorphic_ptr <Base> arr[10]

Множествено наследяване



```

Der d;
Base1* ptr1 = &d;
Base2* ptr2 = &d;
Base3* ptr3 = &d;
  
```

използват специален механизъм, който счита колко трябва да се отместят

$$\begin{aligned}
 &\&d == ptr1 \\
 &ptr1 \neq ptr2 \neq ptr3 \\
 &ptr1 \neq ptr3
 \end{aligned}$$

1) Der()

отговорен за създаването на Base1, Base2 и Base3

```

Der: Base1, Base2, Base3
    A obj1;
    B obj2;
  
```

констр.

дестр.

```

Der(...) : Base1(...) , Base2(...) , Base3(...)
          obj1(...) , obj2(...) { ... }
  
```

перипутирание ги както искане

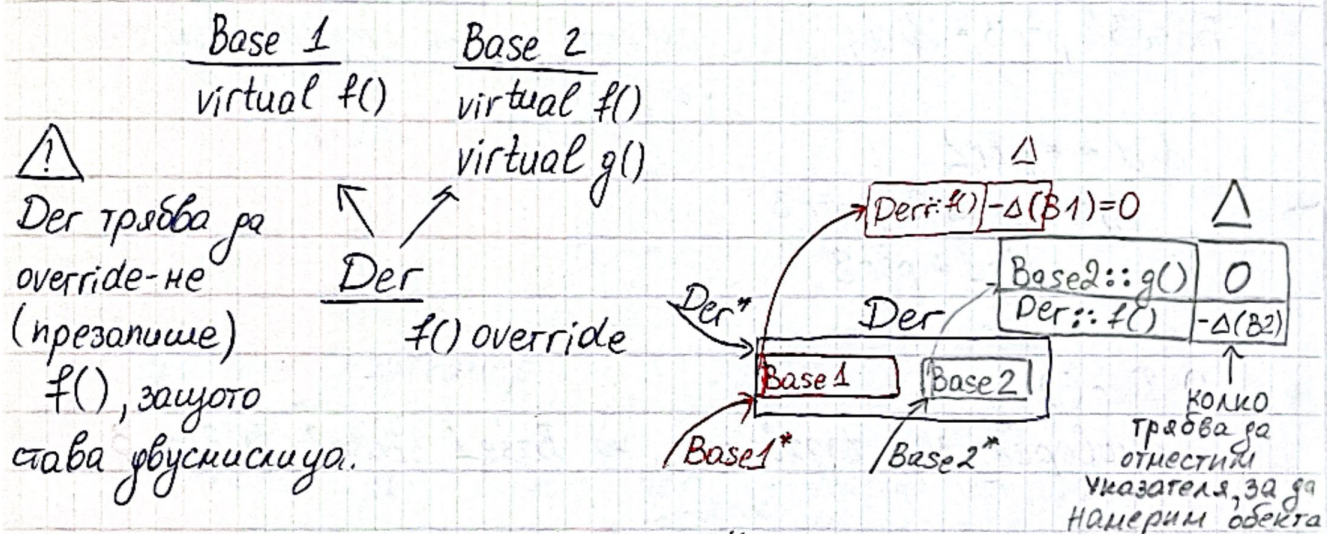
// глере ме това за конст. и дестр.

OP=

```

if (this != &other)
    Base1::op = (other);
    Base2::op = (other);
    Base3::op = (other);
    free()
    copyFrom(other);
    
```

виртуална таблица при множествено наследяване:



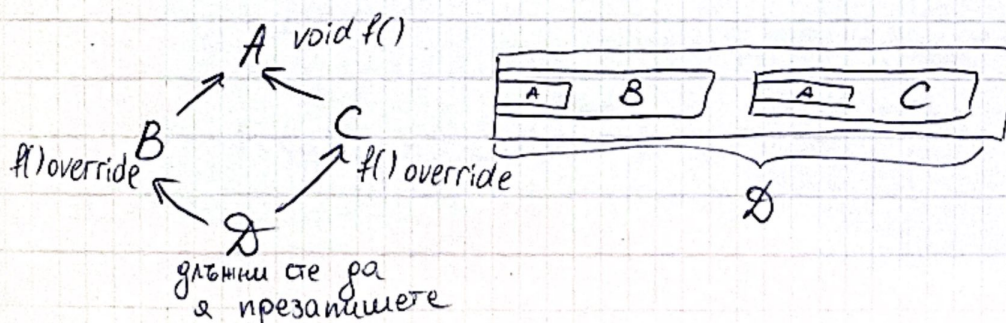
// имаме втори параметър Δ при виртуалната таблица

Δ(Base2) - отнесенето на Base2 от началото на Der

Δ(Base1) = 0

// отнесенето е в байтове

Диамантен "проблем"



$D() \text{ } \cancel{B(...)}, \cancel{C(...)}, \cancel{A(...)}$

отговорки за A

X
↑
Y
↑
Z

$z(): \cancel{Y(...)}, \cancel{X(...)}$

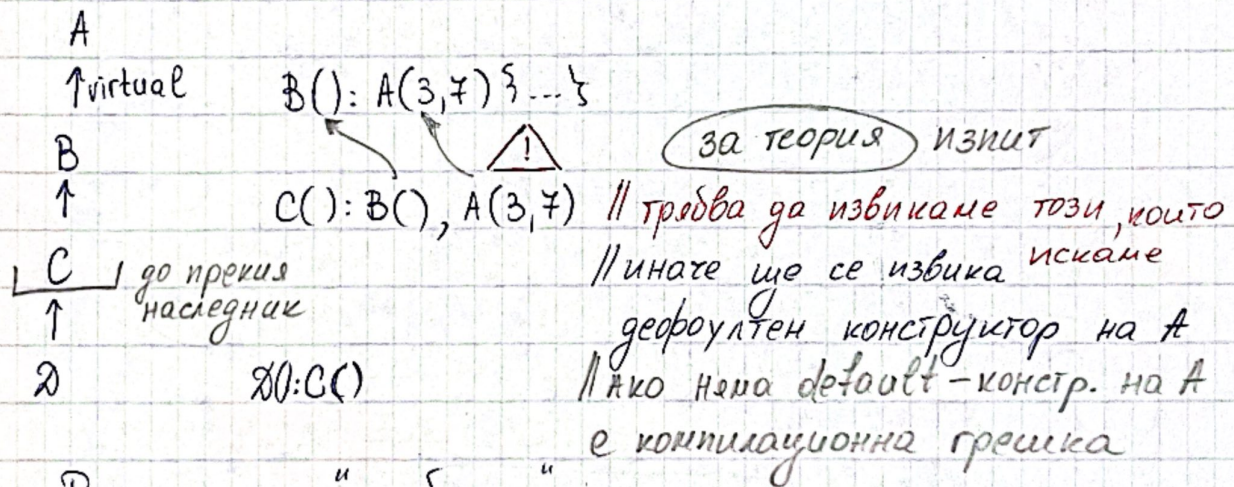
↓
отговорен за X

def: Виртуално наследяване

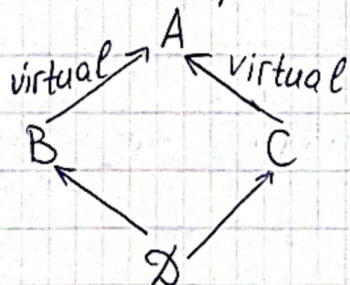
$\text{class } X: \text{virtual } Y \{ \dots \}$

→ Всеки наследник на X е длъжен да каже как да се създаде Y.

→ Прехвърля се отговорността за създаването на Y надолу.



Решение на "проблема"



D е отговорен за създаването на A.

A: [A]

B: [B A]

// sizeof(B) consta n A

C: [C A]

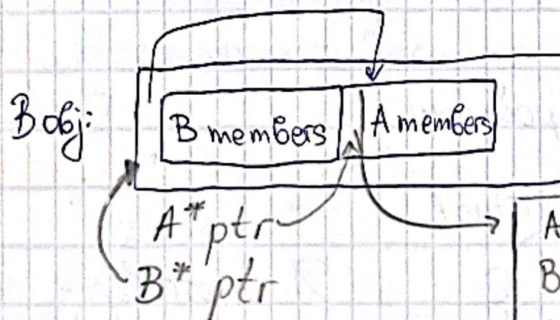
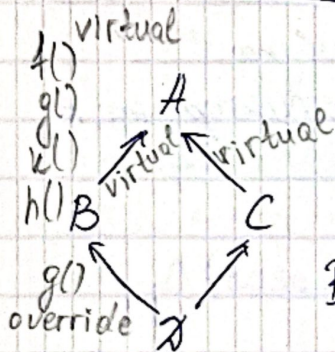
D: [B C A]

// sizeof(D) не е поше
sizeof(B) + sizeof(C)
+ sizeof(A)

~~~D() ~B() ~C() ~A()~~  
~~~A() ~B() ~C() ~D()~~

~~~D() ~A() ~B()~~

~D() ~A() ~C() ~B() // деструктори



|        | $\Delta$     |
|--------|--------------|
| A::f() | 0            |
| B::g() | $-\Delta(A)$ |
| A::k() | 0            |
| A::h() | 0            |